OS Lab

Skill-15

To Parse Combined Redirection, Duplicate File Descriptors, Merge Streams, Verify Ordering, Handle Edge Cases, Test Integrated Outputs.

To Parse Complex Commands, Combine Pipes and Redirections, Resolve Execution Order, Validate Syntax, Generate Execution Plans, Test Scenarios.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>

#define MAX_INPUT 2048
#define MAX_COMMANDS 20
#define MAX_ARGS 50

void execute_command(char *command)
{
   char *args[MAX_ARGS];
   int argc = 0;

   char *input_file = NULL;
   char *output_file = NULL;
   char *append_file = NULL;
   char *error_file = NULL;

   int merge_error = 0;

   char *token = strtok(command, " \t\n");

   while (token != NULL)
   {
      // Input redirection <
      if (strcmp(token, "<") == 0)
      {
         token = strtok(NULL, " \t\n");

         if (token == NULL)
```

```
    {
        fprintf(stderr, "Error: missing input file\n");
        return;
    }

    input_file = token;
}

// Output redirection >
else if (strcmp(token, ">") == 0)
{
    token = strtok(NULL, " \t\n");

    if (token == NULL)
    {
        fprintf(stderr, "Error: missing output file\n");
        return;
    }

    output_file = token;
}

// Append >>
else if (strcmp(token, ">>") == 0)
{
    token = strtok(NULL, " \t\n");

    if (token == NULL)
    {
        fprintf(stderr, "Error: missing append file\n");
        return;
    }

    append_file = token;
}

// stderr redirection 2>
else if (strcmp(token, "2>") == 0)
{
    token = strtok(NULL, " \t\n");

    if (token == NULL)
    {
        fprintf(stderr, "Error: missing error file\n");
```

```
            return;
        }

        error_file = token;
    }

    // Merge stderr with stdout
    else if (strcmp(token, "2>&1") == 0)
    {
        merge_error = 1;
    }

    // Normal argument
    else
    {
        args[argc++] = token;
    }

    token = strtok(NULL, " \t\n");
}

args[argc] = NULL;

if (argc == 0)
    return;

pid_t pid = fork();

if (pid < 0)
{
    perror("fork");
    return;
}

if (pid == 0)
{
    // ========================
    // INPUT <
    // ========================

    if (input_file != NULL)
    {
        int fd = open(input_file, O_RDONLY);
```

```
    if (fd == -1)
    {
        perror("input file");
        exit(EXIT_FAILURE);
    }

    dup2(fd, STDIN_FILENO);
    close(fd);
}

// ========================
// OUTPUT >
// ========================

if (output_file != NULL)
{
    int fd = open(
        output_file,
        O_WRONLY | O_CREAT | O_TRUNC,
        0644
    );

    if (fd == -1)
    {
        perror("output file");
        exit(EXIT_FAILURE);
    }

    dup2(fd, STDOUT_FILENO);
    close(fd);
}

// ========================
// APPEND >>
// ========================

if (append_file != NULL)
{
    int fd = open(
        append_file,
        O_WRONLY | O_CREAT | O_APPEND,
        0644
    );
```

```
    if (fd == -1)
    {
        perror("append file");
        exit(EXIT_FAILURE);
    }

    dup2(fd, STDOUT_FILENO);
    close(fd);
}

// ========================
// STDERR 2>
// ========================

if (error_file != NULL)
{
    int fd = open(
        error_file,
        O_WRONLY | O_CREAT | O_TRUNC,
        0644
    );

    if (fd == -1)
    {
        perror("error file");
        exit(EXIT_FAILURE);
    }

    dup2(fd, STDERR_FILENO);
    close(fd);
}

// ========================
// MERGE 2>&1
// ========================

if (merge_error)
{
    dup2(STDOUT_FILENO, STDERR_FILENO);
}

// Execute command
execvp(args[0], args);
```

```
        perror("execvp");
        exit(EXIT_FAILURE);
    }

    waitpid(pid, NULL, 0);
}


// ======================================
// PIPELINE EXECUTION
// ======================================

void execute_pipeline(char *input)
{
    char *commands[MAX_COMMANDS];
    int command_count = 0;

    char *command = strtok(input, "|");

    while (command != NULL &&
           command_count < MAX_COMMANDS)
    {
        commands[command_count++] = command;
        command = strtok(NULL, "|");
    }

    int pipes[MAX_COMMANDS - 1][2];
    pid_t pids[MAX_COMMANDS];

    // Create pipes
    for (int i = 0; i < command_count - 1; i++)
    {
        if (pipe(pipes[i]) == -1)
        {
            perror("pipe");
            return;
        }
    }

    // Create processes
    for (int i = 0; i < command_count; i++)
    {
        pids[i] = fork();
```

```
if (pids[i] < 0)
{
   perror("fork");
   return;
}

if (pids[i] == 0)
{
   /*
    * Connect input from previous command
    */
   if (i > 0)
   {
      dup2(
         pipes[i - 1][0],
         STDIN_FILENO
      );
   }

   /*
    * Connect output to next command
    */
   if (i < command_count - 1)
   {
      dup2(
         pipes[i][1],
         STDOUT_FILENO
      );
   }

   // Close all pipe descriptors
   for (int j = 0;
       j < command_count - 1;
       j++)
   {
      close(pipes[j][0]);
      close(pipes[j][1]);
   }

   // Parse redirections
   char *args[MAX_ARGS];
   int argc = 0;

   char *input_file = NULL;
```

```
char *output_file = NULL;
char *append_file = NULL;
char *error_file = NULL;

int merge_error = 0;

char *token =
   strtok(commands[i], " \t\n");

while (token != NULL)
{
   if (strcmp(token, "<") == 0)
   {
      token = strtok(NULL, " \t\n");

      if (token != NULL)
         input_file = token;
   }

   else if (strcmp(token, ">") == 0)
   {
      token = strtok(NULL, " \t\n");

      if (token != NULL)
         output_file = token;
   }

   else if (strcmp(token, ">>") == 0)
   {
      token = strtok(NULL, " \t\n");

      if (token != NULL)
         append_file = token;
   }

   else if (strcmp(token, "2>") == 0)
   {
      token = strtok(NULL, " \t\n");

      if (token != NULL)
         error_file = token;
   }

   else if (strcmp(token, "2>&1") == 0)
```

```
    {
        merge_error = 1;
    }

    else
    {
        args[argc++] = token;
    }

    token = strtok(NULL, " \t\n");
}

args[argc] = NULL;

// Input redirection
if (input_file != NULL)
{
    int fd = open(input_file, O_RDONLY);

    if (fd == -1)
    {
        perror("input");
        exit(EXIT_FAILURE);
    }

    dup2(fd, STDIN_FILENO);
    close(fd);
}

// Output redirection
if (output_file != NULL)
{
    int fd = open(
        output_file,
        O_WRONLY | O_CREAT | O_TRUNC,
        0644
    );

    if (fd == -1)
    {
        perror("output");
        exit(EXIT_FAILURE);
    }
```

```
    dup2(fd, STDOUT_FILENO);
    close(fd);
}

// Append redirection
if (append_file != NULL)
{
    int fd = open(
        append_file,
        O_WRONLY | O_CREAT | O_APPEND,
        0644
    );

    if (fd == -1)
    {
        perror("append");
        exit(EXIT_FAILURE);
    }

    dup2(fd, STDOUT_FILENO);
    close(fd);
}

// stderr redirection
if (error_file != NULL)
{
    int fd = open(
        error_file,
        O_WRONLY | O_CREAT | O_TRUNC,
        0644
    );

    if (fd == -1)
    {
        perror("stderr");
        exit(EXIT_FAILURE);
    }

    dup2(fd, STDERR_FILENO);
    close(fd);
}

// Merge stderr with stdout
if (merge_error)
```

```
            {
                dup2(STDOUT_FILENO, STDERR_FILENO);
            }

            if (argc == 0)
            {
                exit(EXIT_SUCCESS);
            }

            execvp(args[0], args);

            perror("execvp");
            exit(EXIT_FAILURE);
        }
    }

    // Parent closes pipes
    for (int i = 0;
         i < command_count - 1;
         i++)
    {
        close(pipes[i][0]);
        close(pipes[i][1]);
    }

    // Wait for all processes
    for (int i = 0;
         i < command_count;
         i++)
    {
        waitpid(pids[i], NULL, 0);
    }
}


// ========================================
// MAIN
// ========================================

int main()
{
    char input[MAX_INPUT];

    while (1)
```

```
{
    printf("combined$ ");
    fflush(stdout);

    if (fgets(
            input,
            sizeof(input),
            stdin) == NULL)
    {
        break;
    }

    input[strcspn(input, "\n")] = '\0';

    if (strlen(input) == 0)
        continue;

    if (strcmp(input, "exit") == 0)
        break;

    /*
     * If | exists,
     * execute as pipeline.
     */
    if (strchr(input, '|') != NULL)
    {
        execute_pipeline(input);
    }
    else
    {
        execute_command(input);
    }
}

return 0;
```

```
lahari@DESKTOP-AIC33F7:~/OSLab$ nano combined.c
lahari@DESKTOP-AIC33F7:~/OSLab$ gcc combined.c -o combined
lahari@DESKTOP-AIC33F7:~/OSLab$ ./combined
combined$ ls existing_file abc.txt > output.txt 2> error.txt
combined$ cat output.txt
combined$ cat error.txt
ls: cannot access 'existing_file': No such file or directory
ls: cannot access 'abc.txt': No such file or directory
combined$ ls existing_file abc.txt > combined.txt 2>&1
combined$ cat combined.txt
ls: cannot access 'existing_file': No such file or directory
ls: cannot access 'abc.txt': No such file or directory
combined$ ls | grep txt > result.txt
combined$ cat result.txt
combined.txt
data.txt
error.txt
file1.txt
files.txt
input.txt
lab.txt
output.txt
combined$ echo "apple" > data.txt
combined$ cat < data.txt | grep apple
"apple"
combined$ cat combined.c | grep int | sort > result.txt
combined$ cat result.txt
```

```
combined$ echo "apple" > data.txt
combined$ cat < data.txt | grep apple
"apple"
combined$ cat combined.c | grep int | sort > result.txt
combined$ cat result.txt
               fprintf(stderr, "Error: missing append file\n");
               fprintf(stderr, "Error: missing error file\n");
               fprintf(stderr, "Error: missing input file\n");
               fprintf(stderr, "Error: missing output file\n");
               int fd = open(
               int fd = open(
               int fd = open(
               int fd = open(input_file, O_RDONLY);
           for (int j = 0;
           int argc = 0;
           int fd = open(
           int fd = open(
           int fd = open(
           int fd = open(input_file, O_RDONLY);
           int merge_error = 0;
       printf("combined$ ");
    for (int i = 0;
    for (int i = 0;
    for (int i = 0; i < command_count - 1; i++)
    for (int i = 0; i < command_count; i++)
    int argc = 0;
    int command_count = 0;
    int merge_error = 0;
    int pipes[MAX_COMMANDS - 1][2];
int main()
combined$ cat combined.c | grep xyz > output.txt 2> error.txt
combined$ exit
lahari@DESKTOP-AIC33F7:~/OSLab$
```